

Solución: Actividad 4 — Análisis Estadístico y Data Mining

Curso: Análisis Estadístico y Data Mining (ISIL, 2026-1)

Tema: Minería de datos aplicada a empresa de tecnología (MarketData Perú)

Alumno: Carlos Gil Carrillo

Contexto general

MarketData Perú vende productos tecnológicos por tres canales: tienda física, página web y redes sociales. La gerencia acumuló datos de clientes, compras, visitas web, reclamos, montos y respuestas a promociones durante un año.

Problemas clave:

- Datos vienen de diferentes áreas con formatos distintos
- Registros con campos vacíos, clientes duplicados, montos mal digitados y categorías inconsistentes
- Las promociones se envían a todos por igual → alto gasto y baja conversión
- No hay segmentos claros de clientes (algunos compran poco, otros frecuentemente, otros solo navegan)
- Se necesita un modelo que prediga qué clientes comprarán laptops
- Se necesita descubrir grupos de clientes con comportamientos similares
- Los directivos necesitan gráficos claros para reuniones ejecutivas
- Un primer modelo obtuvo buenos resultados pero se duda de su estabilidad
- Se debe comparar varios modelos y evitar sobre ajuste

Pregunta 01

Cómo abordar el problema usando minería de datos en Python

La solución se organiza en **cinco fases** del proceso KDD (Knowledge Discovery in Databases)

Etapa	Descripción	Actividades principales
1. Definir el problema	Se entiende el problema y el objetivo del modelo.	- Comprender la necesidad del negocio o investigación - Definir variable objetivo (target) - Identificar tipo de problema (clasificación, regresión, clustering) - Establecer métricas de éxito
2. Preparar los datos	Se limpian y estructuran los datos para su uso.	- Recolección de datos- Limpieza (valores faltantes, duplicados, errores) - Transformación (codificación, escalado, normalización) - División en entrenamiento, validación y prueba

3. Explorar los datos (EDA)	Se analizan los datos para entender su comportamiento.	- Estadística descriptiva - Visualización (histogramas, boxplots, scatter plots) - Detección de outliers - Análisis de correlación - Generación de hipótesis
4. Modelar (Machine Learning)	Se construyen y entrenan los modelos.	- Selección de algoritmos - Entrenamiento del modelo - Ajuste de hiperparámetros - Comparación de modelos - Selección del modelo final
5. Evaluar y comunicar resultados	Se mide el rendimiento y se presentan resultados.	- Evaluación con métricas (Accuracy, F1, RMSE, R^2, etc.) - Matriz de confusión (si aplica) - Interpretación de resultados - Visualización y reporte - Iteración del modelo

Fase 1 — Librerías de Python para cada etapa

El ecosistema de Python para data mining se organiza así:

Etapa	Librería	Para qué sirve
Carga de datos	pandas	Leer CSV, Excel, bases de datos; crear DataFrames
Operaciones numéricas	numpy	Cálculos rápidos con arreglos y matrices
Visualización	matplotlib	Gráficos base (líneas, barras, histogramas, scatter)
Visualización estadística	seaborn	Gráficos estadísticos integrados con pandas (heatmaps, boxplots, distribuciones)
Machine Learning	scikit-learn	Clasificación, clustering, regresión, preprocesamiento, métricas
Ajuste de modelos	scikit-learn	GridSearchCV, RandomizedSearchCV para optimizar hiperparámetros

Instalación:

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

Fase 2 — Preparación de datos (Clase 6)

Antes de modelar, se debe limpiar y transformar el dataset. Según Clase 6, la preparación consume entre 60-80% del tiempo del proyecto.

2.1 Carga y exploración inicial

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Cargar datos de MarketData Perú

```
clientes = pd.read_csv('clientes_marketdata.csv')
compras = pd.read_csv('compras_marketdata.csv')
web = pd.read_csv('visitas_web.csv')
```

Exploración inicial

```
print(clientes.info())
print(clientes.describe())
print(clientes.head())
```

2.2 Problemas de calidad detectados

Según Clase 5 y Clase 6, los problemas descritos en el enunciado son:

Problema	Tipo	Impacto	Solución en Python
Campos vacíos	Valores faltantes	Sesgo si se eliminan, imprecisión si se imputan	fillna(), imputación por media/mediana/moda
Clientes duplicados	Registros repetidos	Sobreponderación artificial	drop_duplicates()
Montos mal digitados	Errores de registro	Outliers que distorsionan análisis	Detección con IQR o Z-score
Categorías inconsistentes	Formatos distintos	Fragmentación del análisis	str.upper(), estandarización de texto

2.3 Limpieza de datos

1. Eliminar duplicados

```
clientes = clientes.drop_duplicates(subset=['id_cliente'])
```

2. Identificar valores faltantes

```
print(clientes.isnull().sum())
```

3. Imputar valores numéricos con la mediana

```
clientes['monto_total'] = clientes['monto_total'].fillna(
    clientes['monto_total'].median()
)
```

4. Imputar valores categóricos con la moda

```
moda_canal = clientes['canal_preferido'].mode()[0]
clientes['canal_preferido'] = clientes['canal_preferido'].fillna(moda_canal)
```

5. Estandarizar nombres de categorías

```
clientes['categoria_producto'] = (  
    clientes['categoria_producto']  
    .str.upper()  
    .str.strip()  
    .str.replace(r'^\w\s', '', regex=True)  
)
```

6. Detectar montos atípicos con IQR

```
Q1 = clientes['monto_total'].quantile(0.25)  
Q3 = clientes['monto_total'].quantile(0.75)  
IQR = Q3 - Q1  
limite_superior = Q3 + 1.5 * IQR  
outliers = clientes[clientes['monto_total'] > limite_superior]  
print(f"Outliers detectados: {len(outliers)}")
```

7. Corregir o eliminar outliers

```
clientes = clientes[clientes['monto_total'] <= limite_superior]
```

2.4 Transformación y codificación de variables

Según Clase 5 y Clase 6, las variables categóricas deben convertirse a numéricas:

```
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Codificar variable categórica (canal de compra)

```
le = LabelEncoder()  
clientes['canal_codificado'] = le.fit_transform(clientes['canal_preferido'])  
# Online=0, Redes=1, Tienda=2
```

Codificar variable binaria (compró o no compró promoción)

```
clientes['compro_promocion'] = clientes['compro_promocion'].map({'Si': 1, 'No': 0})
```

2.5 Normalización de variables numéricas

Según Clase 6, cuando las variables tienen escalas distintas (edad vs. monto), se aplica **StandardScaler** (Z-score):

```
scaler = StandardScaler()  
variables_numericas = ['edad', 'monto_total', 'visitas_web', 'frecuencia_compra']  
clientes[variables_numericas] = scaler.fit_transform(clientes[variables_numericas])
```

Ejemplo de efecto:

Variable	Original	Después de Z-score
Edad (35 años)	35	0.12
Monto total (S/2,500)	2500	-0.85
Visitas web (12)	12	1.34

Todas las variables quedan en escala comparable, evitando que el monto domine al modelo por su magnitud.

Fase 3 — Análisis exploratorio (EDA) y visualización

Antes de modelar, se exploran patrones con gráficos que los directivos pueden entender.

```
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Distribución de montos de compra

```
sns.histplot(clientes['monto_total'], kde=True)
plt.title('Distribución de Montos de Compra')
plt.xlabel('Monto (normalizado)')
plt.show()
```

2. Compras por canal

```
sns.countplot(data=clientes, x='canal_preferido')
plt.title('Clientes por Canal de Compra')
plt.show()
```

3. Relación entre visitas web y compra

```
sns.scatterplot(data=clientes, x='visitas_web', y='compro_promocion')
plt.title('Visitas Web vs. Compra de Promoción')
plt.show()
```

4. Matriz de correlación

```
corr = clientes[['edad', 'monto_total', 'visitas_web', 'frecuencia_compra', 'compro_promocion']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlación entre Variables')
plt.show()
```

Qué buscan los gráficos: - ¿Qué canales generan más ventas? - ¿Las visitas web predicen compra? - ¿Qué variables están correlacionadas con la compra de promociones?

Fase 4 — Modelado: Clasificación y Clustering (Clase 7, Clase 11)

4.1 Clasificación — Predecir si un cliente comprará la promoción de laptops

Algoritmo: Árbol de decisión (explicable, ideal para gerencia) y Regresión Logística (interpretable).

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
```

Variables predictoras y objetivo

```
X = clientes[['edad', 'monto_total', 'visitas_web', 'frecuencia_compra', 'canal_codificado']]
```

```
y = clientes['compro_promocion']
```

```
# División 80% entrenamiento, 20% prueba
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.20, random_state=42, stratify=y  
)
```

```
# Modelo 1: Árbol de Decisión
```

```
arbol = DecisionTreeClassifier(max_depth=5, random_state=42)  
arbol.fit(X_train, y_train)  
y_pred_arbol = arbol.predict(X_test)
```

```
# Modelo 2: Regresión Logística
```

```
logistica = LogisticRegression(random_state=42)  
logistica.fit(X_train, y_train)  
y_pred_logistica = logistica.predict(X_test)
```

```
# Comparar resultados
```

```
print("Árbol de Decisión — Accuracy:", accuracy_score(y_test, y_pred_arbol))  
print("Regresión Logística — Accuracy:", accuracy_score(y_test, y_pred_logistica))
```

Por qué se comparan dos modelos: El área de sistemas advierte no elegir una solución solo porque sea más compleja. Comparar permite fundamentar la decisión.

4.2 Clustering — Descubrir grupos de clientes similares

Algoritmo: K-means (no requiere etiquetas previas, descubre segmentos ocultos).

```
from sklearn.cluster import KMeans  
from sklearn.preprocessing import StandardScaler
```

```
# Seleccionar variables para clustering
```

```
variables_cluster = ['monto_total', 'visitas_web', 'frecuencia_compra']  
X_cluster = clientes[variables_cluster]
```

```
# Escalar (K-means es sensible a la escala)
```

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X_cluster)
```

```
# Aplicar K-means con 3 clusters
```

```
kmeans = KMeans(n_clusters=3, random_state=42)  
clientes['cluster'] = kmeans.fit_predict(X_scaled)
```

```
# Analizar resultados
```

```
print(clientes.groupby('cluster')[variables_cluster].mean())
```

```
,
```

Resultado esperado:

Cluster	Gasto promedio	Visitas web	Frecuencia	Perfil
0	Bajo	Baja	Baja	Clientes ocasionales
1	Medio	Media	Media	Compradores regulares
2	Alto	Alta	Alta	Clientes frecuentes / premium

Acción de marketing: Enviar promoción de laptops prioritariamente al Cluster 2 y Cluster 1, con mensajes diferenciados.

4.3 Visualización de clusters para gerencia

Scatter plot de clusters

```
sns.scatterplot(
    data=clientes,
    x='monto_total',
    y='visitas_web',
    hue='cluster',
    palette='Set2'
)
plt.title('Segmentación de Clientes por Comportamiento')
plt.xlabel('Monto Total (normalizado)')
plt.ylabel('Visitas Web (normalizadas)')
plt.legend(title='Cluster')
plt.show()
```

Fase 5 — Resumen del abordaje

Fase	Qué se hace	Librería principal
1. Preparación	Limpieza, imputación, codificación, normalización	pandas, sklearn.preprocessing
2. Exploración	Histogramas, boxplots, correlaciones	matplotlib, seaborn
3. Clasificación	Predecir compra de promoción (supervisado)	sklearn.tree, sklearn.linear_model
4. Clustering	Segmentar clientes (no supervisado)	sklearn.cluster.KMeans
5. Visualización	Comunicar hallazgos a gerencia	matplotlib, seaborn

Pregunta 02

Cómo evaluar los modelos antes de recomendar su uso

Evaluar un modelo es tan importante como construirlo. Un modelo que funciona bien en datos viejos pero falla en datos nuevos es inútil (sobreajuste). Según Clase 7 y Clase 11, la evaluación se hace con **métricas**

cuantitativas y validación cruzada.

2.1 Métricas de evaluación para clasificación

Métrica	Fórmula	Qué mide	Cuándo usarla
Accuracy	$(VP + VN) / \text{Total}$	Proporción total de aciertos	Clases balanceadas
Precisión	$VP / (VP + FP)$	Confiabilidad de predicciones positivas	Cuando un falso positivo es costoso
Recall	$VP / (VP + FN)$	Cobertura de positivos reales	Cuando es crítico encontrar todos los positivos
F1-Score	$2 \times (\text{Precisión} \times \text{Recall}) / (\text{Precisión} + \text{Recall})$	Equilibrio entre precisión y recall	Clases desbalanceadas

En el caso de MarketData:

- **Accuracy:** ¿Qué porcentaje total de clientes clasificó correctamente?
- **Precisión:** De los que el modelo dijo “comprarán”, ¿cuántos realmente compraron? (importante para no malgastar presupuesto en clientes que no comprarán)
- **Recall:** De los que realmente comprarían, ¿cuántos detectó el modelo? (importante para no perder clientes potenciales)
- **F1-Score:** Balance entre no gastar de más (precisión) y no perder clientes (recall)

`from` sklearn.metrics `import` accuracy_score, precision_score, recall_score, f1_score

Evaluación completa

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precisión:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1-Score:", f1_score(y_test, y_pred))
```

Reporte detallado

```
print(classification_report(y_test, y_pred))
```

2.2 Matriz de confusión

Muestra visualmente los aciertos y errores del modelo:

	Predijo: Compra	Predijo: No Compra
Real: Compra	Verdadero Positivo (VP)	Falso Negativo (FN)
Real: No Compra	Falso Positivo (FP)	Verdadero Negativo (VN)

Interpretación para MarketData:

- **FN (Falsos Negativos):** Clientes que sí comprarían pero el modelo dijo que no
→ se pierde oportunidad de venta
- **FP (Falsos Positivos):** Clientes que NO comprarían pero el modelo dijo que sí
→ se gasta presupuesto en vano

`from` sklearn.metrics `import` ConfusionMatrixDisplay

```
ConfusionMatrixDisplay.from_estimator(arbol, X_test, y_test, cmap='Blues')  
plt.title('Matriz de Confusión — Árbol de Decisión')  
plt.show()
```

2.3 Validación cruzada (K-Fold Cross-Validation)

La validación cruzada evalúa el modelo **múltiples veces** con diferentes particiones, reduciendo el riesgo de que un solo split dé resultados engañosos.

Cómo funciona (K=5):

Fold 1: [Entrenamiento] [Prueba]
Fold 2: [Entrenamiento] [Prueba]
Fold 3: [Entrenamiento] [Prueba]
Fold 4: [Entrenamiento] [Prueba]
Fold 5: [Entrenamiento] [Prueba]

Resultado: Promedio de las 5 evaluaciones

`from` sklearn.model_selection `import` cross_val_score

Validación cruzada con 5 folds

```
scores = cross_val_score(arbol, X, y, cv=5, scoring='accuracy')  
print("Accuracy por fold:", scores)  
print("Accuracy promedio:", scores.mean())  
print("Desviación estándar:", scores.std())
```

Qué busca: Si el accuracy promedio es 0.85 y la desviación es 0.03, el modelo es **estable**. Si la desviación es 0.20, el modelo es **inestable** y no debe usarse en producción.

2.4 Ajuste de hiperparámetros (Grid Search)

Los hiperparámetros son configuraciones externas que se definen antes del entrenamiento (ej: profundidad del árbol, número de vecinos). Según Clase 11, Grid Search prueba todas las combinaciones posibles para encontrar la mejor configuración.

```
from sklearn.model_selection import GridSearchCV
```

```
parametros = {  
    'max_depth': [3, 5, 7, 10],  
    'min_samples_split': [2, 5, 10],  
    'min_samples_leaf': [1, 2, 4]  
}
```

```
grid = GridSearchCV(  
    estimator=DecisionTreeClassifier(random_state=42),  
    param_grid=parametros,  
    cv=5,  
    scoring='accuracy',  
    n_jobs=-1  
)  
grid.fit(X_train, y_train)
```

```
print("Mejores hiperparámetros:", grid.best_params_)  
print("Mejor accuracy:", grid.best_score_)
```

Evaluar el mejor modelo en test

```
mejor_modelo = grid.best_estimator_  
y_pred_final = mejor_modelo.predict(X_test)  
print("Accuracy en test:", accuracy_score(y_test, y_pred_final))
```

2.5 Comparación de modelos (Clase 11)

El jefe de analítica pide comparar varios modelos antes de decidir:

```
from sklearn.ensemble import RandomForestClassifier  
from sklearn.neighbors import KNeighborsClassifier
```

```
modelos = {  
    'Árbol de Decisión': DecisionTreeClassifier(max_depth=5, random_state=42),  
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),  
    'Regresión Logística': LogisticRegression(random_state=42),  
    'KNN': KNeighborsClassifier(n_neighbors=5)  
}
```

```
resultados = {}  
for nombre, modelo in modelos.items():  
    scores = cross_val_score(modelo, X, y, cv=5, scoring='accuracy')  
    resultados[nombre] = {  
        'Accuracy promedio': round(scores.mean(), 4),  
        'Desviación estándar': round(scores.std(), 4)  
    }  
}
```

```
df_resultados = pd.DataFrame(resultados).T
print(df_resultados)
```

Resultado esperado:

Modelo	Accuracy promedio	Desviación estándar
Árbol de Decisión	0.82	0.04
Random Forest	0.87	0.03
Regresión Logística	0.80	0.05
KNN	0.78	0.06

Decisión: Random Forest tiene el mejor accuracy y menor variabilidad → es el candidato para producción.

2.6 Criterios de decisión final

Antes de recomendar un modelo al área de marketing, se verifican:

Criterio	Pregunta	Si falla
Accuracy	¿El modelo acierta más del 80%?	Revisar features o probar otro algoritmo
Estabilidad	¿La desviación estándar es baja (<0.05)?	El modelo es inestable, no usar en producción
Sobreajuste	¿Accuracy en train similar a test?	Si hay brecha grande → sobreajuste, simplificar modelo
Interpretabilidad	¿Puede gerencia entender la decisión?	Si es caja negra, usar modelo más simple
Costo de error	¿Cuánto cuesta un falso positivo vs. falso negativo?	Ajustar umbral de decisión según prioridad

Verificar sobreajuste

```
train_accuracy = accuracy_score(y_train, arbol.predict(X_train))
test_accuracy = accuracy_score(y_test, arbol.predict(X_test))
```

```
print(f"Accuracy en entrenamiento: {train_accuracy:.4f}")
print(f"Accuracy en prueba: {test_accuracy:.4f}")
print(f"Brecha: {abs(train_accuracy - test_accuracy):.4f}")
```

Si la brecha > 0.10, hay sobreajuste

```
if abs(train_accuracy - test_accuracy) > 0.10:
```

```
    print("⚠️ Posible sobre ajuste detectado. Simplificar el modelo.")
```

```
else:
```

```
    print("✅ Modelo estable.")
```

Resumen: Plan de acción para MarketData Perú

Fase	Acción	Herramienta	Beneficio
1. Preparar	Limpiar duplicados, imputar faltantes, normalizar escalas	pandas, StandardScaler	Datos confiables para modelar
2. Explorar	Histogramas, correlaciones, boxplots	matplotlib, seaborn	Entender patrones antes de modelar
3. Clasificar	Predecir compra de laptops	DecisionTree, LogisticRegression	Campañas dirigidas, menor gasto
4. Segmentar	Descubrir grupos de clientes	KMeans	Campañas diferenciadas por perfil
5. Evaluar	Validación cruzada, comparación de modelos	cross_val_score, GridSearchCV	Modelo estable y confiable
6. Comunicar	Gráficos claros para gerencia	seaborn, matplotlib	Decisiones basadas en evidencia

Resultado esperado: - Campañas de laptops dirigidas a clientes con alta probabilidad de compra → **menor gasto, mayor conversión** - Segmentos de clientes identificados → **estrategias diferenciadas por grupo** - Modelo validado y estable → **confianza para usar en producción**

Referencias

- **Clase 5 — Minería de Datos:** Definición, tipos (clasificación, clustering, asociación), proceso KDD
- **Clase 6 — Preparación de Datos:** Limpieza, imputación, normalización Z-score y Min-Max
- **Clase 7 — Algoritmos de Minería de Datos:** Árboles de decisión, SVM, K-NN, K-means, métricas de evaluación
- **Clase 9 — Instalación y Entornos:** Librerías de Python, configuración del entorno
- **Clase 11 — Minería de Datos en Python:** Scikit-learn, validación cruzada, Grid Search, visualización
- **Clase 12 — Análisis Integrado R y Python:** Pipelines de datos, integración de flujos
- Documentación oficial:
 - Pandas: <https://pandas.pydata.org/docs/>
 - Scikit-learn: <https://scikit-learn.org/stable/>
 - Matplotlib: <https://matplotlib.org/>
 - Seaborn: <https://seaborn.pydata.org/>